

The way to declare lists in go: (This declaration called *slice*)

```
var taskList = []string{
    "1. Watch the crash course of Go",
    "2. Build network automation tools with Go",
    "3. Build Microservices Applications With gRPC",
    "4. Build my Applications using Go",
}
```

Slices in Go

- ▶ Slices are dynamically-sized and more flexible than arrays



If we declare variables without using them, then the result is:

```
C:\Tests\Go\From-Youtube-01>go run main.go
# command-line-arguments
.\main.go:16:2: declared and not used: task05

C:\Tests\Go\From-Youtube-01>
```

Comments

- ▶ Text within the source code that are ignored and is not executed by the compiler
- ▶ Usually used for explanations or documentation of the code, making it easier for developers to understand what code does



If we define the size of slice, then it's Array:

Arrays in Go



- ▶ Arrays have a fixed size
(size = how many elements the array can hold)

```
var variableName = [size]variable_type
```

We declare arrays: `var list = [size]variable_type {}` *// it is constant size*

- **Ex:** `var tasks = [20]string {"...", "..."}`

Slice is an abstraction of an Array

To print multiple variables, using `fmt`:

```
fmt.Println("*** Task Of Items ***")
fmt.Println("Tasks", taskSlice)
```



Loop Types in Go

Basic for loop	▶ A loop that executes a block of code a specific number of times
for loop with condition	▶ A loop that continues executing as long as a specified condition is true
for range loop	▶ A loop that iterates over elements in slices, arrays or maps

for Range Loop

► Provides both the index and value during each iteration

`taskItems = ["Crash Course", "Full Course", "Reward"]`

task



1st Iteration:

"Crash Course"



2nd Iteration:

"Full Course"



3rd Iteration:

"Reward"

Note: *for* and *range* is reserved words in Golang:

```
for index, task := range taskSlice {
    fmt.Println("The index is:", index, ", and task is: ", task)
}
```

Using *_* to *ignore variables*, especially for index of *for && range loops*:

Here we use temp variables with for-loop (task-variable)

Blank Identifier



► To ignore a variable you don't want to use

► Unlike other languages, in Go you need to make unused variables explicit

Common Placeholders

Here are some common format specifiers used with `fmt.Printf`:



Placeholder	Description	Example Usage
<code>%d</code>	Decimal integer	<code>fmt.Printf("%d", 42)</code>
<code>%f</code>	Floating-point number	<code>fmt.Printf("%f", 3.14)</code>
<code>%s</code>	String	<code>fmt.Printf("%s", "Hello")</code>
<code>%v</code>	Default format for any value	<code>fmt.Printf("%v", myVar)</code>
<code>%T</code>	Type of the value	<code>fmt.Printf("%T", myVar)</code>
<code>%p</code>	Pointer address	<code>fmt.Printf("%p", &myVar)</code>
<code>%t</code>	Boolean value	<code>fmt.Printf("%t", true)</code>

Printf = Print formatted data

- ▶ It takes a **template string** that contains the text that needs to be formatted
- ▶ Plus some **placeholders** that tells the `fmt` function how to format the variable passed in

`%s`
`%v`
`%T`
`%p`
`%t`

We Can use format string using `fmt`:

```
for index, task := range taskSlice {  
    fmt.Printf("%d. %s\n", index+1, task)  
}
```

Functions

- ▶ Encapsulate code into own container (= function). Which logically belong together!
- ▶ Like variable name, you should give a function a **descriptive name**
- ▶ Call the function by its name, whenever you want to execute this block of code



```
func greetUser {  
    fmt.Println("Welcome to our conference")  
}  
  
greetUser() // this executes the above code
```

2 Main Types of Scope

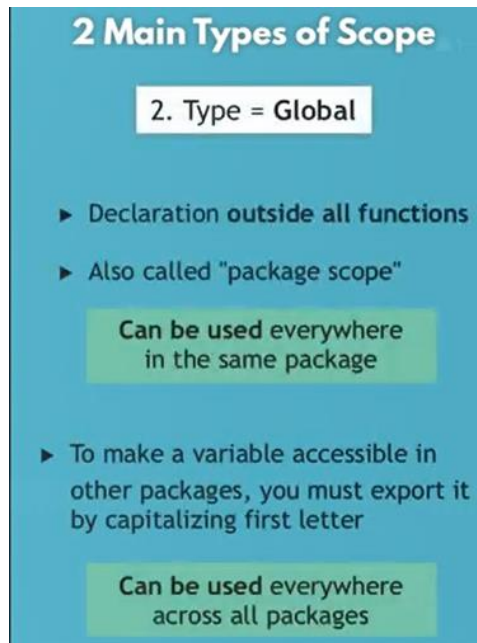
1. Type = Local

- ▶ Declaration within function
- ▶ Also called "function scope"

Can be used only
within that function

- ▶ Declaration within block
(e.g. for, if-else)
- ▶ Also called "block scope"

Can be used only
within that block



To define function with params:

```
func printTasks(taskItems []string) {  
    fmt.Println("##### Welcome to our Todolist App! #####")  
  
    for index, task := range taskItems {  
        fmt.Printf("%d. %s\n", index+1, task)  
    }  
}
```

To make the function return a value, we must change the signature of the function:

Note: If the function return many values not one, we must use (), ex: ([]string, ...etc)

```
func addTask(taskItems []string, newTask string) []string {  
    var updatedTaskItems = append(taskItems, newTask)  
  
    return updatedTaskItems  
}
```

To Create Http Server:

```
package main

import "fmt"
import "net/http"

func main() {
    // in this way we create handle for / without register it in ListenAndServe

    http.HandleFunc("/", helloLoka)
    http.ListenAndServe("localhost:8080", nil);
}

func helloLoka(responseWriter http.ResponseWriter, request *http.Request) {
    var greeting = "Hi, My Name is Jafar Loka. Welcome to Todo project";
    fmt.Fprintln(responseWriter, greeting);
}
*****

func printTasks(responseWriter http.ResponseWriter, taskItems []string) {

    fmt.Println("List of my Todos")

    for index, task := range taskItems {
        // fmt.Println(index+1, ".", task)
        fmt.Fprintf(responseWriter, "%d. %s\n", index+1, task)
    }
}
*****
```